



CANCER EPIGENETICS COURSE

BASICS OF GOOGLE COLAB, R PROGRAMMING LANGUAGE

Presenter: Tran Duc Phuc

21st December 2025

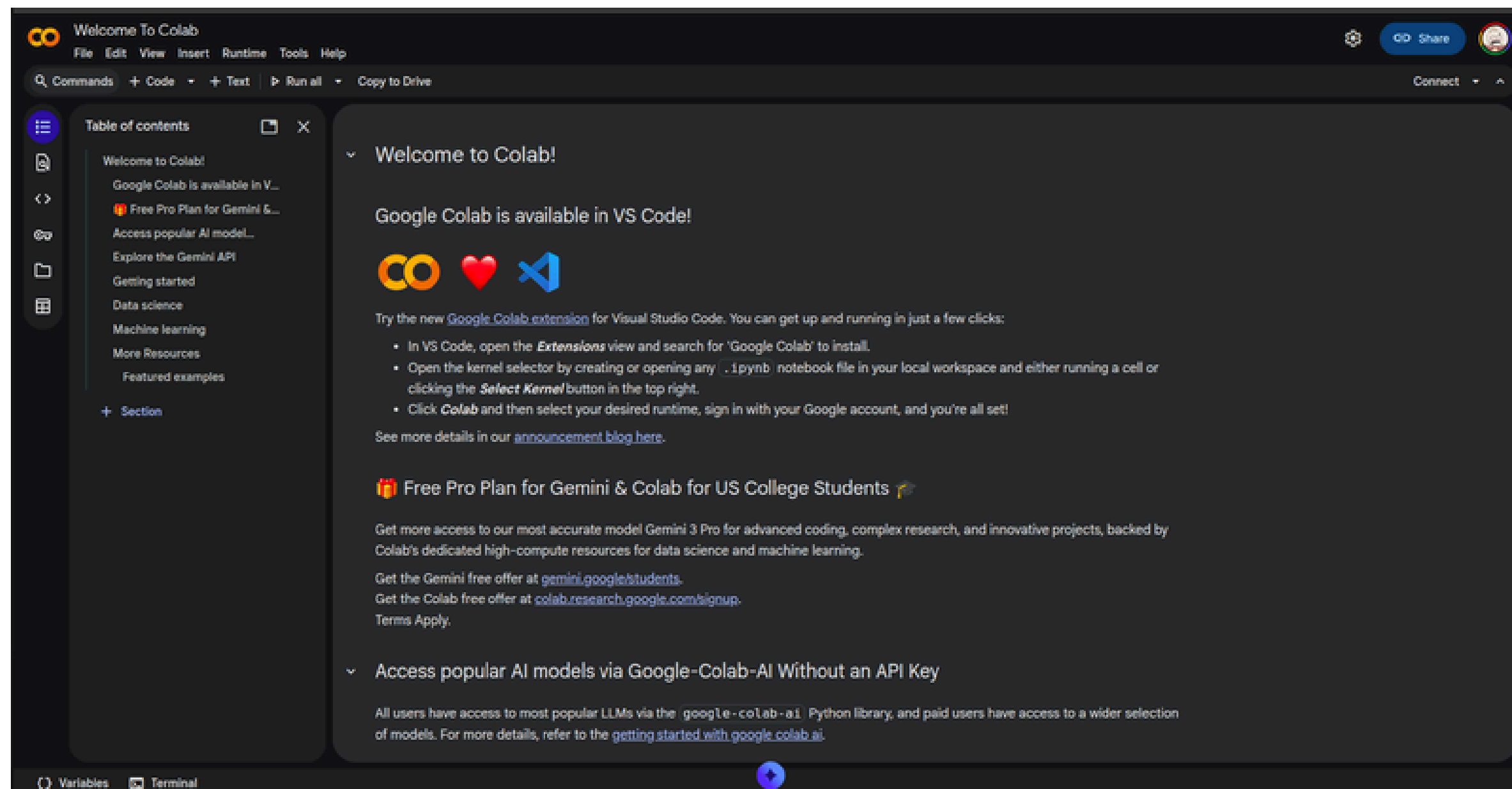
Google

colab



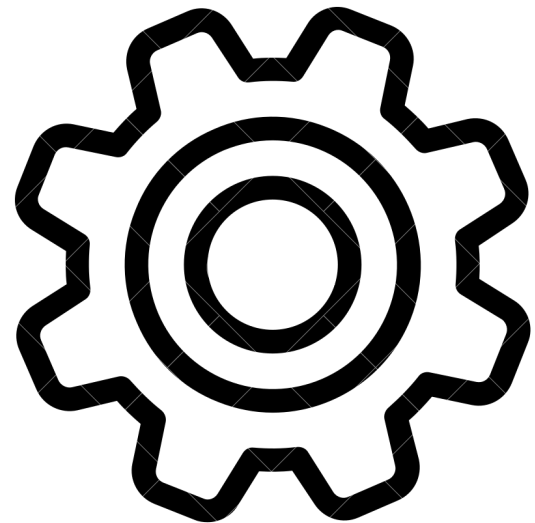
INTRODUCTION TO GOOGLE COLAB

- Free, cloud-based tool that lets you write and run code
- “Google Docs for Code”
- runs entirely in your web browser—no software installation required



INTRODUCTION TO GOOGLE COLAB

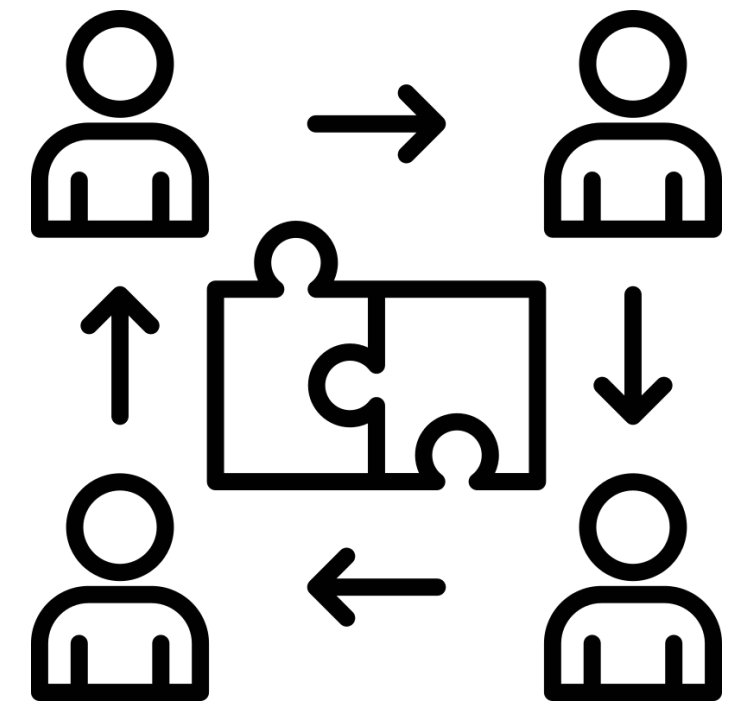
Zero Setup



Cloud Saving



Collaboration



R BASICS



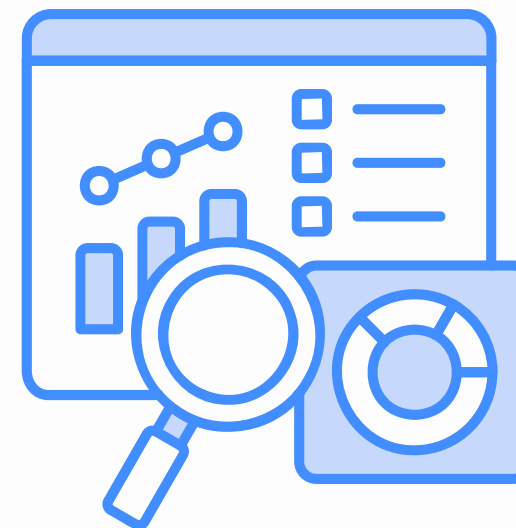
Link to google colab: <https://colab.research.google.com/drive/1UKARP5im6-lIDUl2risx8qfTIYFwKnz5?usp=sharing>

R Programming Language

- Open-source programming language
- Main functions:
 - Statistical analysis
 - Data visualization



Main function



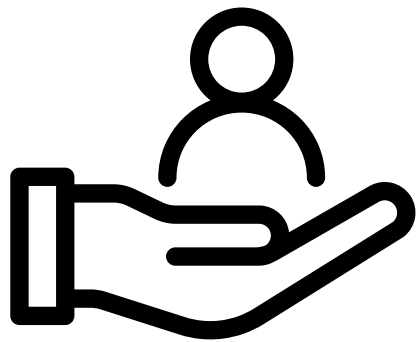
Statistical analysis



Visualization

R programming language

Customizable



- Highly customize
- Multiple packages

Free open-source



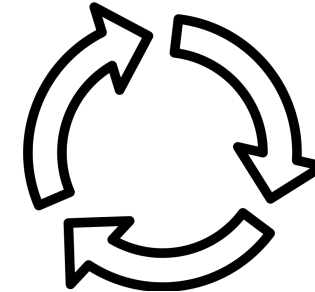
- Completely free

Active community support



- Multiple forums, community
- Help debugging

Reproducible



- Ex: using 1 script for many studies

Simple



- Simpler than the majority CLI software

Running R

- **Interactive Mode:** Commands are typed at a prompt (`>`), and results are displayed immediately
- **IDEs:** Most programmers use environments like RStudio rather than a simple terminal

```
phuc@phuc-TDP:~$ R

R version 4.5.2 (2025-10-31) -- "[Not] Part in a Rumble"
Copyright (C) 2025 The R Foundation for Statistical Computing
Platform: x86_64-pc-linux-gnu

R is free software and comes with ABSOLUTELY NO WARRANTY.
You are welcome to redistribute it under certain conditions.
Type 'license()' or 'licence()' for distribution details.

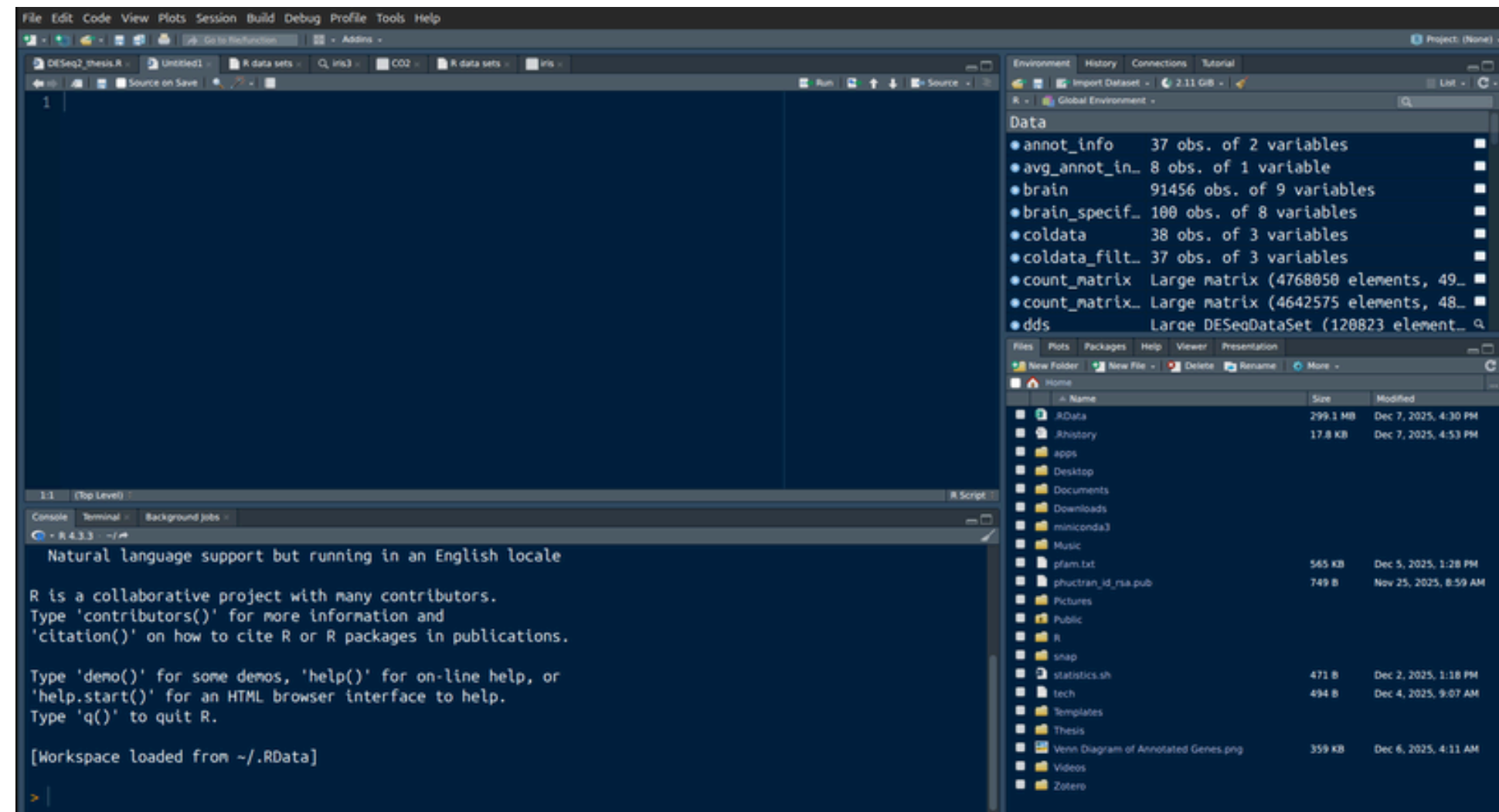
Natural language support but running in an English locale

R is a collaborative project with many contributors.
Type 'contributors()' for more information and
'citation()' on how to cite R or R packages in publications.

Type 'demo()' for some demos, 'help()' for on-line help, or
'help.start()' for an HTML browser interface to help.
Type 'q()' to quit R.

[Previously saved workspace restored]

> 
```

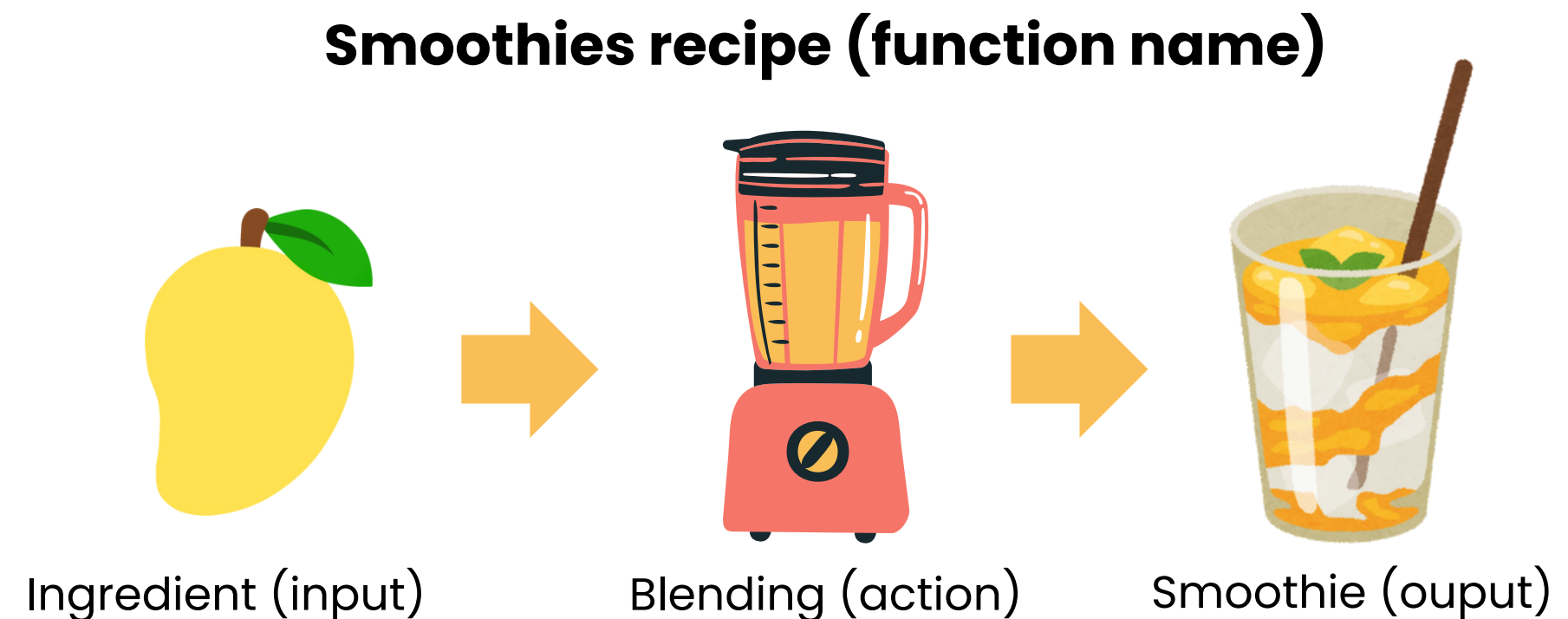


FUNCTIONS

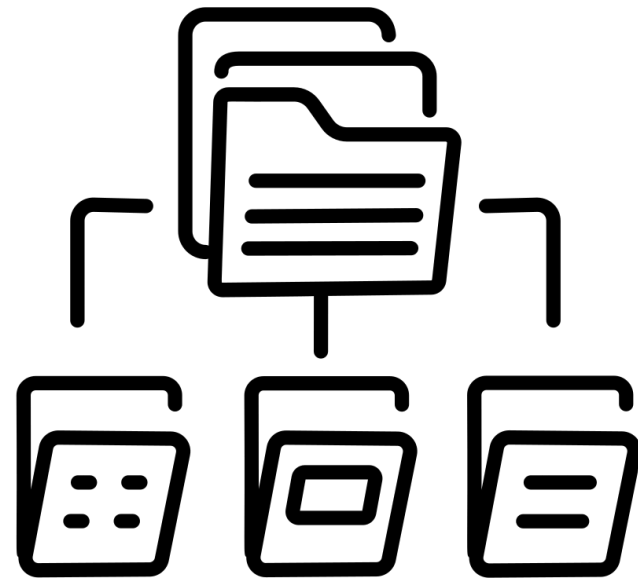
Link to google colab: <https://colab.research.google.com/drive/1UKARP5im6-lIDUl2risx8qfTIYFwKnz5?usp=sharing>

Functions

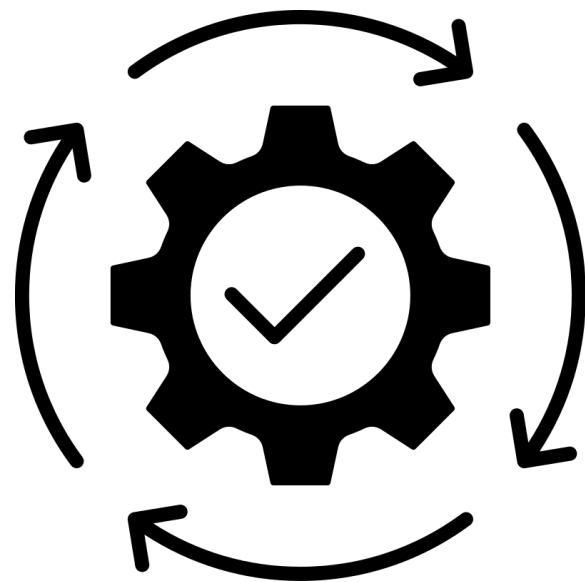
- Function: a group of instructions that takes inputs, computes other values, and returns a result.
- R has default functions:
 - `mean ()`
 - `sum ()`
 - `read.table ()`
 - ...
- R allows defining functions:
 - `function_name ← function (input)`
 {action
 return (output)}
- Why define a new function?
 - Highly reproducible
 - Don't have to write many lines of code



Why using functions?



Organization



Automation



Without functions

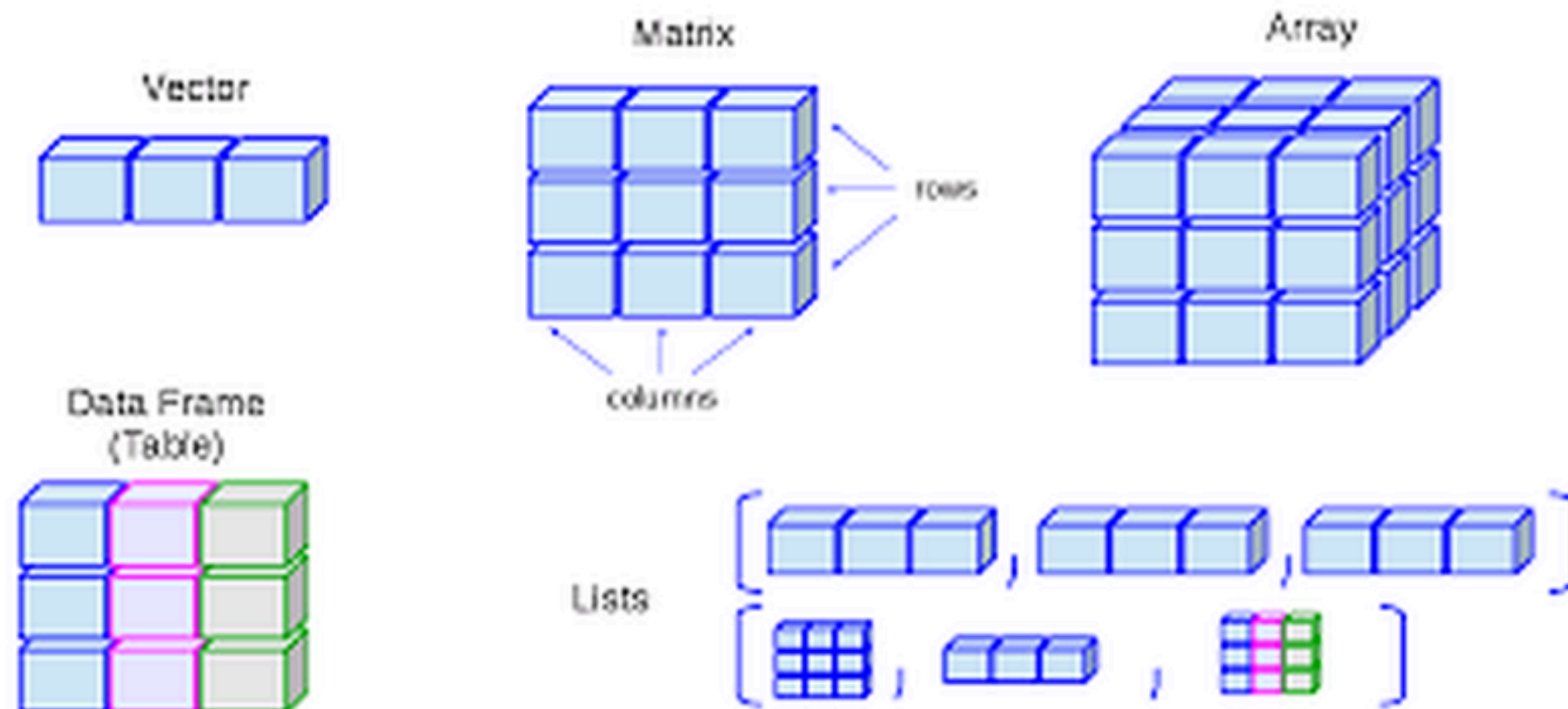
```
> price <- 100
> tax <- price * 0.10
> total <- price + tax
> total
[1] 110
>
> price <- 1000
> tax <- price * 0.10
> total <- price + tax
> total
[1] 1100
>
> price <- 100000
> tax <- price * 0.10
> total <- price + tax
> total
[1] 110000
```



With functions

```
> calculate_total <- function(price) {
+   tax <- price * 0.10
+   total <- price + tax
+   return(total)
+ }
> calculate_total (100)
[1] 110
> calculate_total (1000)
[1] 1100
> calculate_total (1000000)
[1] 1100000
```

Data structure



Packages

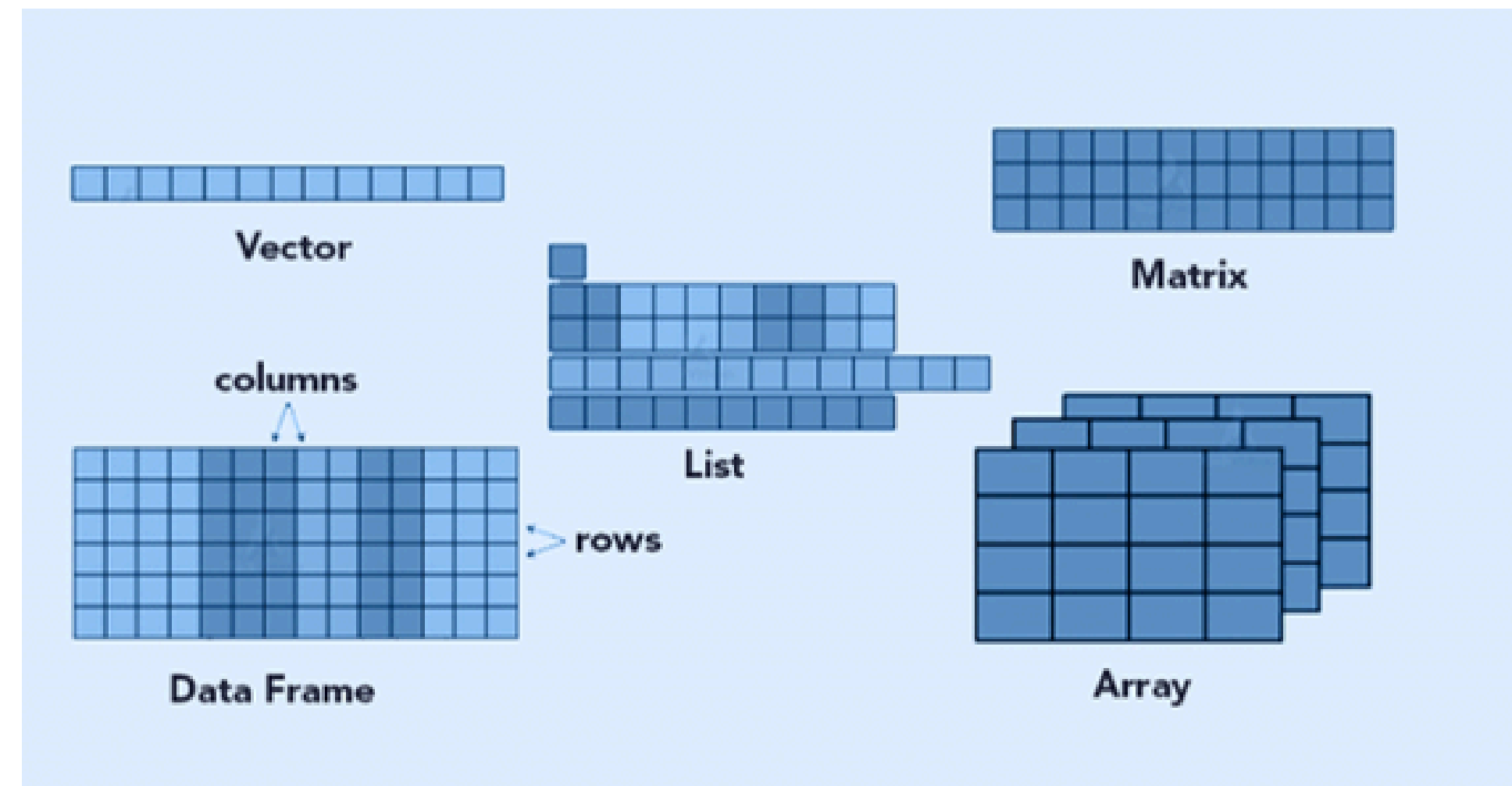


DATA STRUCTURE

Link to google colab: <https://colab.research.google.com/drive/1UKARP5im6-lIDUl2risx8qfTIYFwKnz5?usp=sharing>

Data structure

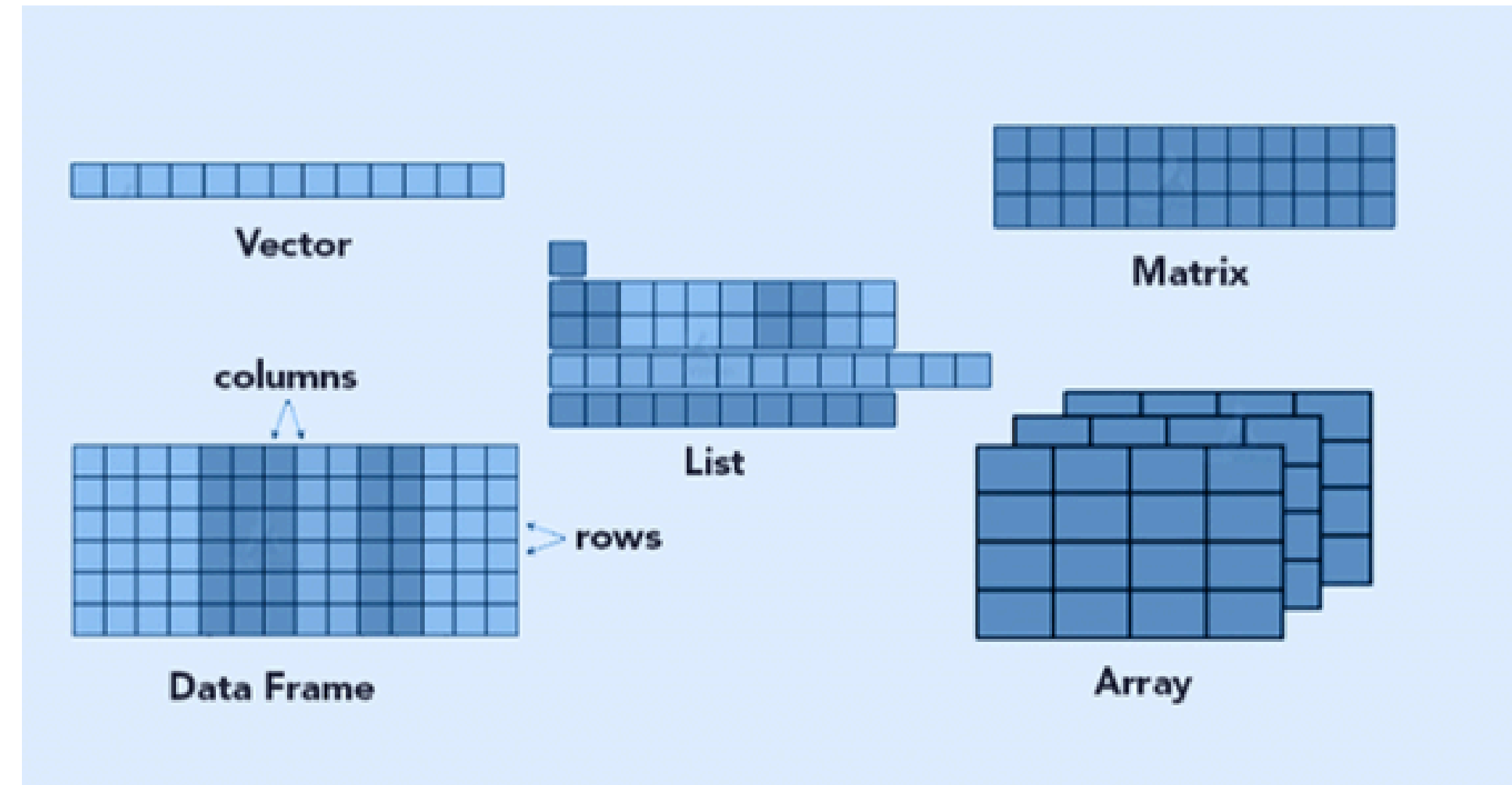
- Data structure is a container for storing and organizing data, designed to handle everything from simple lists of numbers to complex tables of mixed data types.
- Dimensionality (1D, 2D, or n-dimensional)
- Homogeneity (whether all elements must be the same type).



	Homogeneous	Heterogeneous
1-D	Atomic vector	List*
2-D	Matrix	Data Frame
n-D	Array	

Vector

- The vector type is the heart of R
- The elements of a vector must all have the same mode, or data type. (Homogeneous)
- Dimension: 1-D
- Other data structures are just vectors with more or fewer attributes:
 - Matrices are vectors with rows and columns
 - Scalars (individual numbers) are vectors with a single component

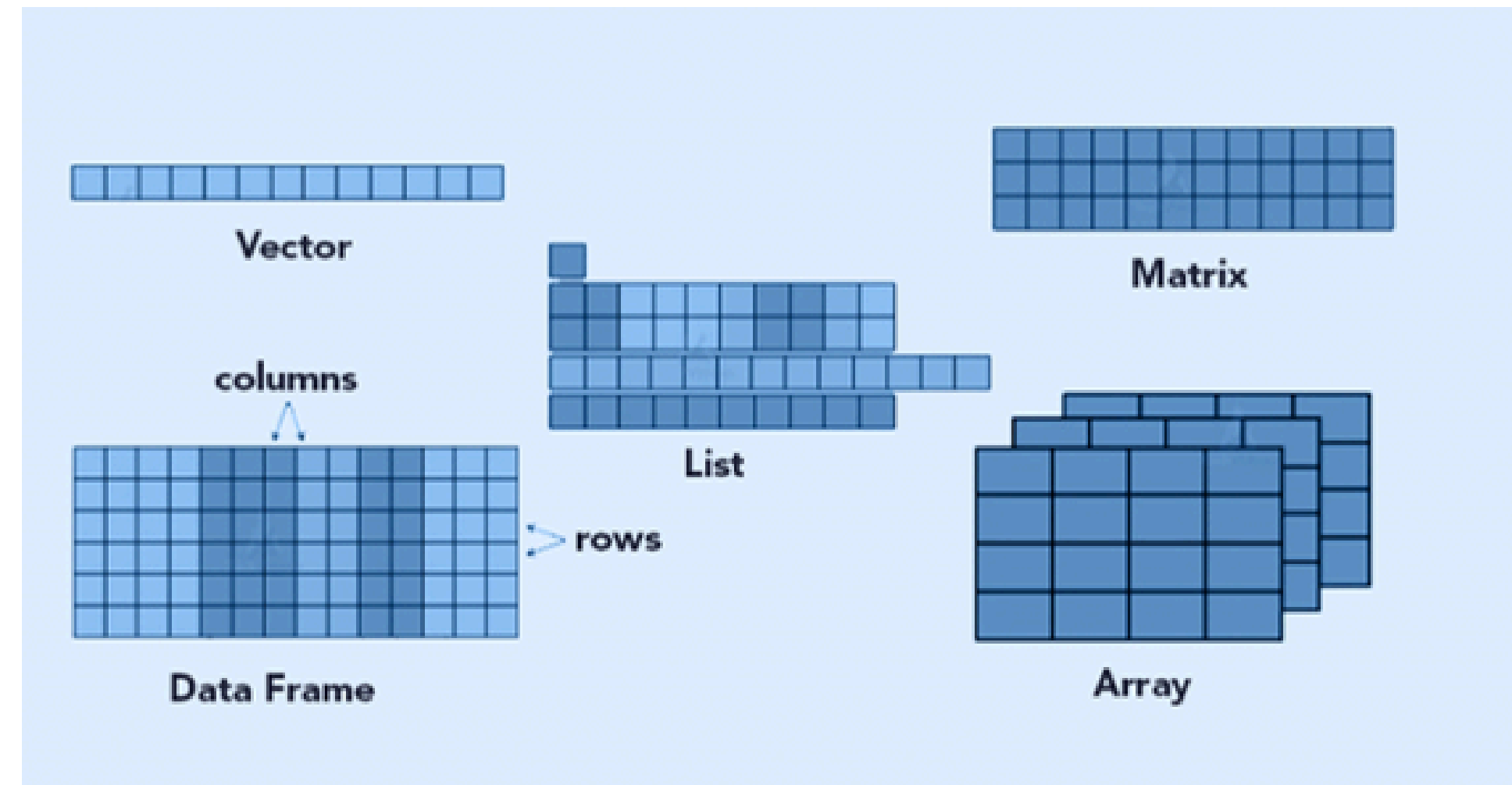


Vectors in R

Index	→	1	2	3	4	5	6	7	8	9	10
Values	→	10	20	30	40	50	60	70	80	90	100

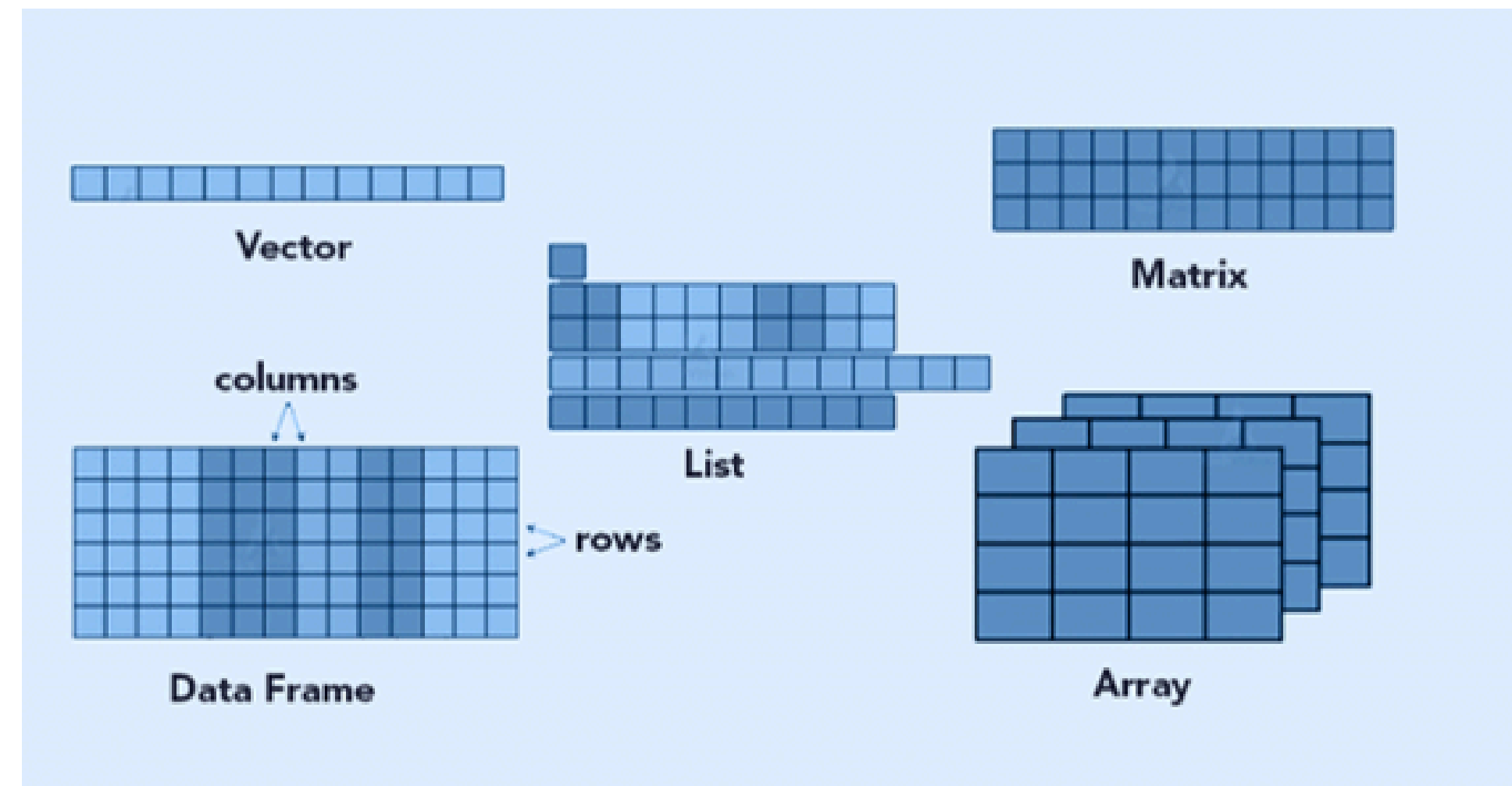
List

- List structure can combine objects of different types
- Lists are referred to as recursive vectors
- Dimension: 1-D
- Heterogeneous: contains many data types, modes



Matrix

- A matrix is a rectangular array of numbers.
- A matrix is a vector, but with two additional attributes: rows and columns
- Vectors are not one-column or one-row matrices.
- Dimension: 2-D
- **Homogeneous**



Data frame

- Data frames are matrices with different data types, modes
- Data frames are lists with rows and columns
- Dimension: 2-D
- **Heterogeneous**

		Column →		
		0	1	2
Row ↓	0	2	3	5
	1	7	14	21
	2	1	3	5

PACKAGES

Link to google colab: <https://colab.research.google.com/drive/1UKARP5im6-lIDUl2risx8qfTIYFwKnz5?usp=sharing>

Packages

- Packages store groups of related pieces of software
- Some packages are loaded automatically
- Does not load all available packages automatically



Homework

Homework 1:

1. Vector: Create a numeric vector named `grades` containing five exam scores. (9, 8, 5, 4, 7)
2. Matrix: Construct a 5x2 matrix called `attendance` where each row represents a student and columns represent "Days Present" and "Days Absent".
3. List: Create a list named `student_profile` that stores a student's name (character), their grades vector, and their attendance matrix. (any name)
4. Data Frame: Build a data frame named `class_summary` that includes columns for `Student_Name` and `Final_Grade` (calculated as the mean of a vector).
5. Access the grades vector from within your `student_profile` list using the `$` operator and calculate its average.

Homework

Homework 2:

1. Vector: Define a character vector of `item_names` and a numeric vector of `unit_prices`.
2. Matrix: Create a matrix called `store_stock` with 3 rows (stores) and 4 columns (products), filling it with random inventory counts.
3. Data Frame: Combine your `item_names` and `unit_prices` into a data frame named `price_list`.
4. List: Create an `inventory_packet` list that contains the `price_list` data frame and the `store_stock` matrix.
5. Add a new column to the `price_list` data frame called `discounted_price` by multiplying the `unit_prices` vector by 0.9.

Homework

Homework 3:

1. Vector: Create a character vector of five cities and a numeric vector of their populations.
2. Matrix: Build a 5x5 distance matrix where each element $m[i,j]$ represents the distance between two cities.
3. Data Frame: Create a data frame `city_info` from your cities and populations vectors. Create a second data frame `city_coords` with cities and their "Latitude/Longitude".
4. List: Combine both data frames and the distance matrix into a master list called `region_map`.
5. Use the `merge()` function to combine `city_info` and `city_coords` by the "City" column into a single comprehensive data frame